🏦 **Project: Simple Bank Management System**

🎯 **Goal:**

Create a program that allows users to:

- Create an account
- Deposit money
- Withdraw money
- Check balance
- Exit the system

💻 **Code:**

```
print("🏦 Welcome to Simple Bank Management System 🏦")


accounts = {}  # Dictionary to store account details


while True:
    print("\n------ MENU ------")
    print("1. Create Account")
    print("2. Deposit Money")
    print("3. Withdraw Money")
    print("4. Check Balance")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ")

    if choice == "1":
```

```
    name = input("Enter your name: ")
    acc_no = input("Enter account number: ")
    accounts[acc_no] = {"Name": name, "Balance": 0}
    print("✅ Account created successfully!")

elif choice == "2":
    acc_no = input("Enter account number: ")
    if acc_no in accounts:
        amount = float(input("Enter amount to deposit: "))
        accounts[acc_no]["Balance"] += amount
        print("💰 Amount deposited successfully!")
    else:
        print("❌ Account not found!")

elif choice == "3":
    acc_no = input("Enter account number: ")
    if acc_no in accounts:
        amount = float(input("Enter amount to withdraw: "))
        if amount <= accounts[acc_no]["Balance"]:
            accounts[acc_no]["Balance"] -= amount
            print("💸 Withdrawal successful!")
        else:
            print("❌ Insufficient balance!")
    else:
        print("❌ Account not found!")
```

```
elif choice == "4":
    acc_no = input("Enter account number: ")
    if acc_no in accounts:
        print(f"👤 Name: {accounts[acc_no]['Name']}")
        print(f"💰 Balance: ₹{accounts[acc_no]['Balance']}")
    else:
        print("❌ Account not found!")

elif choice == "5":
    print("👏 Thank you for using our bank system!")
    break
else:
    print("⚠️ Invalid choice! Try again.")
```

🧠 **How It Works:**

1. Uses a **while True loop** for continuous menu display.
2. Uses a **dictionary** to store multiple accounts (acc_no → details).
3. Handles deposit, withdrawal, and balance operations using loops & conditions.
4. Ends when the user chooses option 5 (Exit).

🧩 **Project: Student Grade Management System**

🎯 **Goal:**

Create a program that:

- Takes details of multiple students (name, marks).
- Calculates their average.
- Decides their grade.
- Displays a full report.

💻 **Code:**

```
print("🎓 Student Grade Management System 🎓")


students = []  # list to store student data


while True:
    name = input("\nEnter student name (or 'stop' to finish): ")
    if name.lower() == 'stop':
        break


    total_marks = 0
    subjects = int(input("Enter number of subjects: "))


    for i in range(subjects):
        marks = float(input(f"Enter marks for subject {i+1}: "))
```

```
        total_marks += marks

    avg = total_marks / subjects

    # Grade calculation
    if avg >= 90:
        grade = 'A+'
    elif avg >= 75:
        grade = 'A'
    elif avg >= 60:
        grade = 'B'
    elif avg >= 40:
        grade = 'C'
    else:
        grade = 'F'

    students.append({"Name": name, "Average": avg, "Grade": grade})

# Final Report
print("\n📋 Final Student Report:")
print("-" * 35)
print("Name\t\tAverage\tGrade")
print("-" * 35)
for s in students:
    print(f"{s['Name']}\t\t{s['Average']:.2f}\t{s['Grade']}")
```

```
print("-" * 35)

print("✅ Report Generated Successfully!")
```

🧠 **How It Works:**

1. Uses a **while loop** to repeatedly take student names and marks.
2. Uses a **for loop** to take multiple subject marks.
3. Calculates average and assigns grades.
4. Stores all data in a list of dictionaries.
5. Prints a final **student report card table**.

🍔 **Project: Restaurant Billing System**

🎯 **Goal:**

Create a program that:

- Shows a food menu
- Lets customers order multiple items
- Calculates total bill with tax
- Prints a final receipt

💻 **Code:**

```
print("🍽️ Welcome to Python Restaurant 🍽️")


menu = {
    "Burger": 80,
    "Pizza": 120,
    "Pasta": 100,
    "Coffee": 60,
    "Sandwich": 70
}


order = {}  # store ordered items


while True:
    print("\n------ MENU ------")
    for item, price in menu.items():
        print(f"{item} : ₹{price}")
```

```
    choice = input("\nEnter item name to order (or 'done' to finish): ").capitalize()


    if choice.lower() == "done":
        break
    elif choice in menu:
        qty = int(input(f"Enter quantity of {choice}: "))
        if choice in order:
            order[choice] += qty
        else:
            order[choice] = qty
        print(f"✅ {choice} added to your order.")
    else:
        print("⚠️ Item not on the menu!")


# Calculate Bill
print("\n📜 Your Order Summary 📜")
print("-" * 30)
total = 0
for item, qty in order.items():
    price = menu[item] * qty
    total += price
    print(f"{item}\t x{qty}\t = ₹{price}")


tax = total * 0.05
```

```
grand_total = total + tax

print("-" * 30)

print(f"Subtotal: ₹{total}")

print(f"GST (5%): ₹{tax:.2f}")

print(f"Total Bill: ₹{grand_total:.2f}")

print("-" * 30)

print("💖 Thank you! Visit again! 💖")
```

🧠 **How It Works:**

1. Uses **while True loop** to take multiple orders.
2. Stores orders in a dictionary (item → quantity).
3. Calculates total bill and adds 5% tax.
4. Prints a **neat bill summary** with all items.

📚 **Project: Library Management System**

🎯 **Goal:**

Create a program that:

- Adds new books
- Displays all books
- Allows borrowing and returning of books
- Works in a continuous menu using loops

💻 **Code:**

```
print("📚 Welcome to Python Library Management System 📚")


library = []  # list to store book details


while True:
    print("\n------ MENU ------")
    print("1. Add Book")
    print("2. Show All Books")
    print("3. Borrow Book")
    print("4. Return Book")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ")

    # 1️⃣ Add a new book
    if choice == "1":
```

```
    title = input("Enter book title: ")
    author = input("Enter author name: ")
    library.append({"Title": title, "Author": author, "Available": True})
    print(f"✅ '{title}' added successfully!")

# 2️⃣ Show all books
elif choice == "2":
    if len(library) == 0:
        print("📭 No books in library yet!")
    else:
        print("\n📘 Available Books:")
        print("-" * 40)
        for i, book in enumerate(library, start=1):
            status = "✅ Available" if book["Available"] else "❌ Borrowed"
            print(f"{i}. {book['Title']} by {book['Author']} - {status}")

# 3️⃣ Borrow a book
elif choice == "3":
    title = input("Enter book title to borrow: ")
    found = False
    for book in library:
        if book["Title"].lower() == title.lower():
            found = True
            if book["Available"]:
                book["Available"] = False
```

```
            print(f"📕 You borrowed '{book['Title']}'.")
        else:
            print("❌ Sorry, that book is already borrowed!")
        break
    if not found:
        print("⚠️ Book not found!")

# 4️⃣ Return a book
elif choice == "4":
    title = input("Enter book title to return: ")
    found = False
    for book in library:
        if book["Title"].lower() == title.lower():
            found = True
            if not book["Available"]:
                book["Available"] = True
                print(f"📗 '{book['Title']}' returned successfully!")
            else:
                print("⚠️ That book was not borrowed.")
            break
    if not found:
        print("⚠️ Book not found!")

# 5️⃣ Exit
elif choice == "5":
```

```
    print("👏 Thank you for using the Library System!")
    break
else:
    print("⚠️ Invalid choice! Try again.")
```

🧠 **How It Works:**

1. Uses **a list of dictionaries** to store book details.
2. Uses a **while True loop** to show the menu continuously.
3. Books can be **added, borrowed, returned, or displayed**.
4. Loops ensure you can keep using the system until you exit.

🏨 **Project: Hotel Booking System**

🎯 **Goal:**

Create a program that allows users to:

- View available rooms
- Book a room
- Cancel a booking
- View all bookings
- Exit the system

💻 **Code:**

```
print("🏨 Welcome to Python Hotel Booking System 🏨")


rooms = {
    101: {"Type": "Single", "Price": 1000, "Booked": False},
    102: {"Type": "Double", "Price": 1800, "Booked": False},
    103: {"Type": "Deluxe", "Price": 2500, "Booked": False},
    104: {"Type": "Suite", "Price": 3500, "Booked": False},
}


bookings = {}  # store guest details


while True:
    print("\n------ MENU ------")
    print("1. View Available Rooms")
    print("2. Book a Room")
```

```
print("3. Cancel Booking")
print("4. View All Bookings")
print("5. Exit")

choice = input("Enter your choice (1-5): ")

# 1️⃣ View available rooms
if choice == "1":
    print("\n🏨 Available Rooms:")
    print("-" * 40)
    for room_no, info in rooms.items():
        if not info["Booked"]:
            print(f"Room {room_no} - {info['Type']} - ₹{info['Price']}/night")
    print("-" * 40)

# 2️⃣ Book a room
elif choice == "2":
    room_no = int(input("Enter room number to book: "))
    if room_no in rooms:
        if not rooms[room_no]["Booked"]:
            name = input("Enter your name: ")
            days = int(input("Enter number of days: "))
            total = rooms[room_no]["Price"] * days
            rooms[room_no]["Booked"] = True
            bookings[room_no] = {"Name": name, "Days": days, "Total": total}
```

```
        print(f"✅ Room {room_no} booked successfully for {name}!")
        print(f"💰 Total Bill: ₹{total}")
    else:
        print("❌ Room already booked!")
else:
    print("⚠️ Invalid room number!")

# 3️⃣ Cancel booking
elif choice == "3":
    room_no = int(input("Enter room number to cancel: "))
    if room_no in bookings:
        rooms[room_no]["Booked"] = False
        del bookings[room_no]
        print(f"🗑️ Booking for room {room_no} cancelled successfully.")
    else:
        print("⚠️ No booking found for that room!")

# 4️⃣ View all bookings
elif choice == "4":
    if len(bookings) == 0:
        print("📭 No current bookings.")
    else:
        print("\n📋 Current Bookings:")
        print("-" * 50)
        for room_no, details in bookings.items():
```

```
        print(f"Room {room_no} → {details['Name']} | Days: {details['Days']} | ₹{details['Total']}")
    print("-" * 50)

# 5️⃣ Exit
elif choice == "5":
    print("👋 Thank you for using the Hotel Booking System!")
    break

else:
    print("⚠️ Invalid choice! Please try again.")
```

🧠 **How It Works:**

1. Uses **dictionaries** to store room data and booking info.
2. Uses a **while loop** for the main menu.
3. Handles booking, canceling, and displaying data using nested loops and conditions.
4. Keeps running until the user exits.

💵 **Project: ATM Simulator**

🎯 **Goal:**

Create a program that simulates ATM functions:

- Check balance
- Deposit money
- Withdraw money
- Exit the system

💻 **Code:**

```
print("💳 Welcome to Python ATM Simulator 💳")


balance = 5000  # starting balance


while True:
    print("\n------ MENU ------")
    print("1. Check Balance")
    print("2. Deposit Money")
    print("3. Withdraw Money")
    print("4. Exit")

    choice = input("Enter your choice (1-4): ")

    # 1️⃣ Check Balance
    if choice == "1":
        print(f"💰 Your current balance is: ₹{balance}")
```

```
# 2️⃣ Deposit Money
elif choice == "2":
    amount = float(input("Enter amount to deposit: "))
    if amount > 0:
        balance += amount
        print(f"✅ ₹{amount} deposited successfully!")
        print(f"New balance: ₹{balance}")
    else:
        print("⚠️ Invalid amount!")

# 3️⃣ Withdraw Money
elif choice == "3":
    amount = float(input("Enter amount to withdraw: "))
    if amount <= balance and amount > 0:
        balance -= amount
        print(f"💸 ₹{amount} withdrawn successfully!")
        print(f"Remaining balance: ₹{balance}")
    elif amount > balance:
        print("❌ Insufficient balance!")
    else:
        print("⚠️ Invalid amount!")

# 4️⃣ Exit
elif choice == "4":
```

```
    print("👋 Thank you for using Python ATM!")
    break

else:
    print("⚠️ Invalid choice! Please try again.")
```

🧠 **How It Works:**

1. Starts with a **default balance**.
2. Uses a **while loop** for continuous ATM menu.
3. Uses **if-else** conditions for options.
4. Updates balance dynamically after each transaction.
5. Ends only when the user chooses to exit.